# Understanding Neural Network Design: How Depth and Width Affect Multilayer Perceptron (MLP) Performance

Chukwufumnaya Favour Nonum GitHub Link: MyProfile ID: 24159030

## 1. Introduction

When working with neural networks, one of the first practical questions that comes up is: how complex should the model be? In particular, how many layers should we use, and how many neurons should each layer contain?

This tutorial focuses on answering that question using a Multilayer Perceptron (MLP). Rather than just explaining the theory only, I explore how changing the depth (number of layers) and width (number of neurons per layer) affects model performance in practice.

The goal is to provide an understanding that can help others make better design decisions when building neural networks.

## 2. What is a Multilayer Perceptron?

A Multilayer Perceptron (MLP) is a type of neural network made up of layers of interconnected neurons. It consists of:

- An **input layer** (receives data)
- One or more **hidden layers** (learn patterns)
- An **output layer** (produces prediction

Each neuron applies a weighted sum of inputs followed by an activation function, which introduces non-linearity into the model. Without activation functions, the network would behave like a linear model regardless of its depth, limiting its ability to learn complex patterns.

In this implementation, the **ReLU** (Rectified Linear Unit) activation function is used by default in the hidden layers of the **MLPClassifier**, which computes **(f(x) = \max (0, x)).** This choice is widely used in practice due to its efficiency and strong empirical performance.

## 3. Why Depth and Width Matter

Two key design choices in a Multilayer Perceptron are:

- **Depth:** Number of hidden layers
- **Width:** Number of neurons in each layer

**Depth (Number of Layers):** Increasing depth allows the model to learn more abstract representations. Earlier layers may learn simple features, while deeper layers combine them into more complex representations.

However, deeper networks are harder to train and may suffer from issues such as unstable gradients or overfitting if not properly regularised.

**Width (Number of Neurons):** Increasing width allows each layer to capture more variation in the data. A wider network can approximate more complex functions, but excessive width can lead to overfitting and unnecessary computational cost.

## 4. Dataset and Setup

The Breast Cancer Wisconsin dataset was used in this tutorial.

It contains **569** samples with **30** numerical features describing characteristics of cell nuclei derived from medical images.

The task is to classify tumours as either **malignant (0)** or **benign (1)**.

The dataset was split into training and testing sets, and input feature scaling was applied using **standardisation**. This ensures that all features are on a similar scale, which improves the performance of neural networks.

## 5. Experiment 1: Effect of Depth (number of hidden layers)

To study the effect of depth, I fixed the width at **16** and increased the depth from **1** through **4**.

As displayed in Figure 1, the results showed that:

- A single hidden layer achieved the best performance, **~0.982**
- Adding more layers did not improve accuracy
- Deeper models actually reduced performance

This suggests that increasing depth introduced unnecessary complexity for this dataset and did not provide additional benefits.

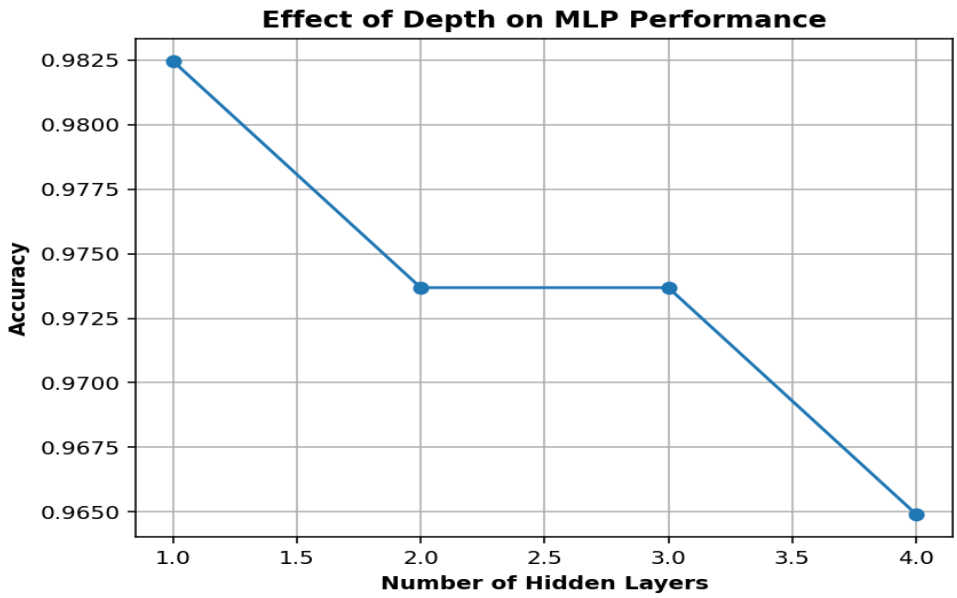


***Figure 1:*** Plot of Effect of Depth on MLP Performance

## Experiment 2: Effect of Width (number of neurons)

Next, I investigated the effect of width by keeping the depth fixed at **2 layers** and increased the number of neurons per layer using **8, 16, 32, 64 neurons** respectively.

As displayed in Figure 2, the results showed that:

- Increasing neurons improved performance up to **32 neurons**
- The best accuracy of **~0.991** was achieved at **32 neurons**
- Increasing to **64 neurons** reduced performance significantly

This indicates that while increasing model capacity can improve learning, too many neurons can lead to poorer generalisation.

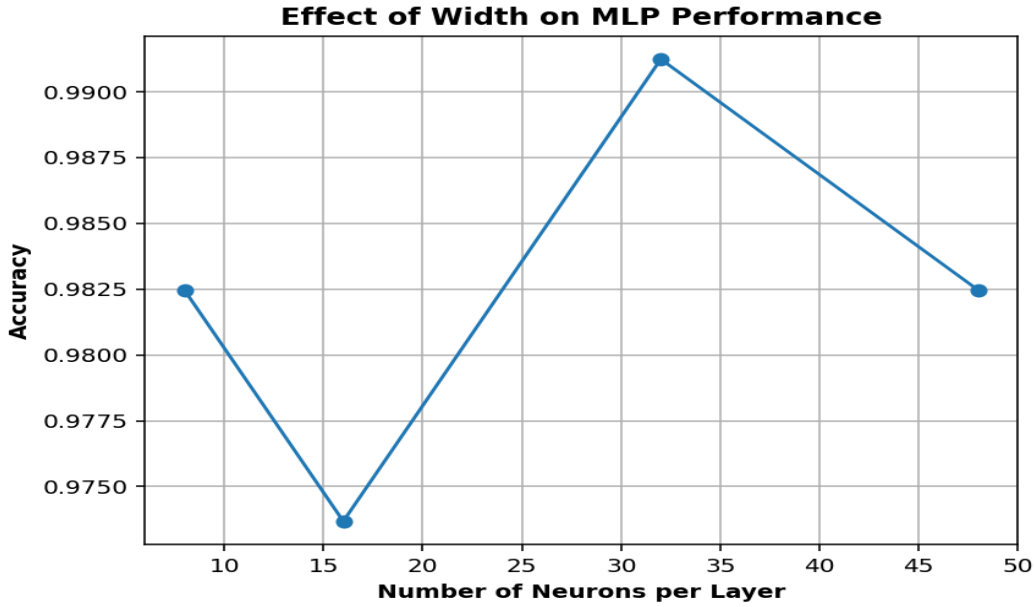


***Figure 2:*** Plot of Effect of Width on MLP Performance

## 6. <u>Overfitting Analysis</u>

To better understand the behaviour of larger models, I compared training and testing accuracy (*Figure 3*).

The results showed that:

- Larger models achieved nearly perfect training accuracy
- Testing accuracy decreased for the largest model **(64 neurons)**

This is a clear sign of overfitting, where the model learns the training data too well but fails to generalise to new data.

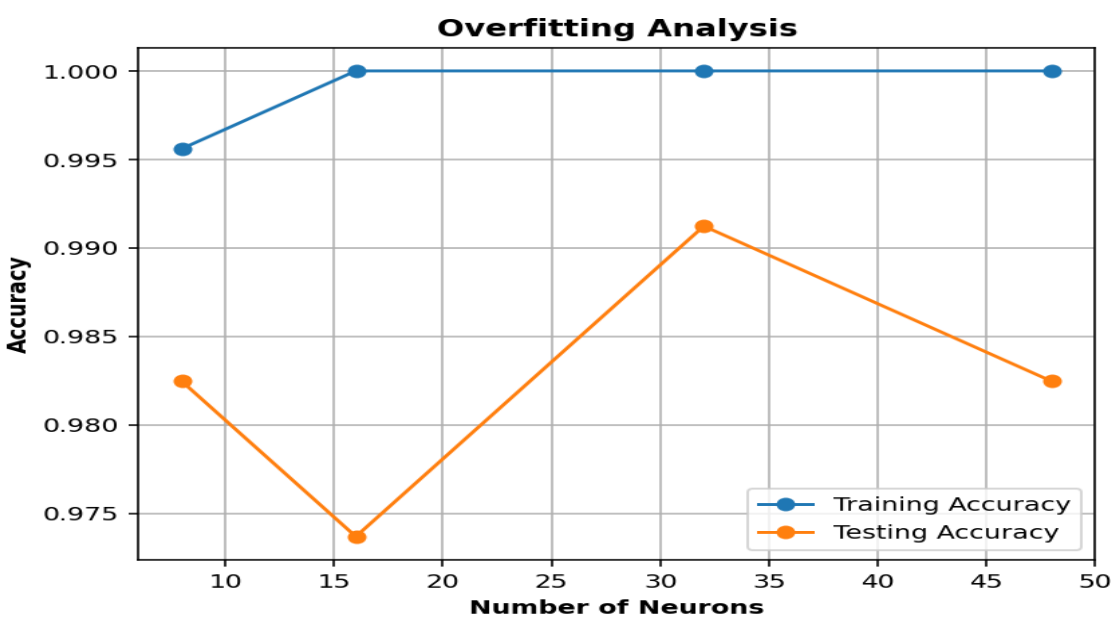


***Figure 3:*** Plot of Overfitting

## 7. Improving the Model to Address Overfitting:

While the earlier experiments clearly showed that larger models led to overfitting, it is important to go a step further and explore how this issue can be addressed in practice.

To improve the model, I adjusted the architecture and introduced simple regularisation techniques. First, the number of neurons was reduced to a moderate level of **32 neurons** as the results showed that increasing beyond this point did not improve performance. In addition, **L2 regularisation** was applied to discourage overly large weights, and **early stopping** was used to prevent the model from continuing training longer than necessary.

After applying these changes, the model showed a better balance between training and testing performance (see *figure 4*). The gap between training and testing accuracy was reduced, indicating that the model was no longer simply memorising the training data.

This demonstrates that controlling model complexity and applying regularisation are effective ways to improve generalisation in neural networks.

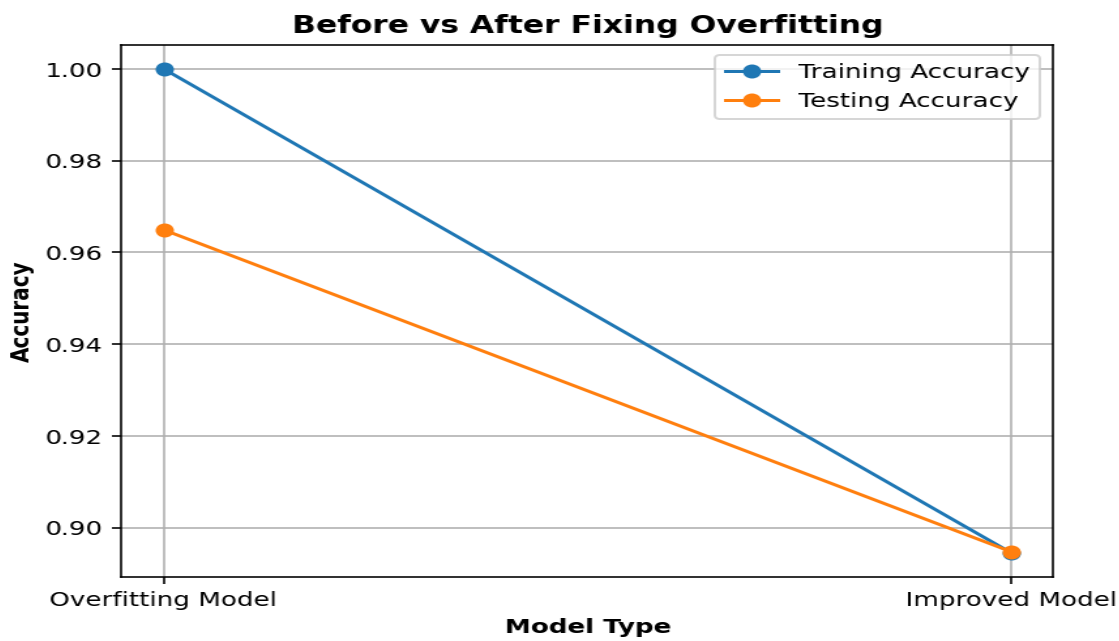


***Figure 4***: Overfitting Model vs Improved Model

## 8. Discussion

The experiments highlight that increasing model complexity does not always lead to better performance.

- Deeper models did not improve results for this dataset
- Wider models improved performance up to a point
- Excessive complexity led to overfitting

The best results were achieved using a moderate model size combined with regularisation.

## 9. Critical Reflection

While the experiments provided useful insights, there are limitations:

- This analysis is based on a single dataset, so the results may not apply to all types of data. Different datasets may require different model architectures.
- The dataset size **(569 samples**) and complexity (due to many feature variables) may influence the observed trends, limiting generalisability. This may have contributed to the overfitting observed.
- In addition, only depth and width were explored in detail. Other factors such as learning rate, different activation functions, and optimisation methods were not investigated.

Finally, the evaluation was based mainly on accuracy, which may not capture all aspects of model performance.

## 10. Conclusion

In this tutorial, I explored how the depth and width of a **Multilayer Perceptron (MLP)** affect its performance.

From the experiments, it is clear that increasing the complexity of a neural network does not always improve performance. A simple model with a moderate number of neurons performed better than deeper or wider models.

While larger models achieved very high training accuracy, they did not generalise well to unseen data.

After applying regularisation and reducing model complexity, the model became more balanced and achieved better overall performance. This shows the importance of finding the right balance when designing neural networks.

## **References**:


Agarap, A.F. (2017) On breast cancer detection: An application of machine learning algorithms on the Wisconsin diagnostic dataset. Available at: https://arxiv.org/abs/1711.07831

Glorot, X. and Bengio, Y. (2010) 'Understanding the difficulty of training deep feedforward neural networks', Proceedings of AISTATS.

Goodfellow, I., Bengio, Y. and Courville, A. (2016) Deep Learning. MIT Press.

He, K. et al. (2015) 'Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification', Proceedings of ICCV.

Kingma, D.P. and Ba, J. (2014) 'Adam: A method for stochastic optimisation', arXiv preprint arXiv:1412.6980.

Nguyen, A. et al. (2021) 'An analysis of state-of-the-art activation functions for supervised deep neural networks', arXiv preprint arXiv:2104.02523.

scikit-learn (2024) MLPClassifier documentation. Available at: https://scikit-learn.org

scikit-learn (2023) Neural network models (supervised). Available at: https://scikit-learn.org

Wolberg, W.H., Street, W.N. and Mangasarian, O.L. (1992) Breast cancer Wisconsin dataset.

Géron, A. (2019) Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow. O'Reilly Media.

Xu, B. et al. (2015) 'Empirical evaluation of rectified activations in convolutional networks', arXiv preprint arXiv:1505.00853.

Agarap, A.F. (2018) 'Deep learning using rectified linear units (ReLU)', arXiv preprint arXiv:1803.08375.